

Homework 2 - Frequent Itemsets and Association Rules (Apriori)

Data Mining

AHMAD AL KHATEEB OMAR ALMASSRI
ahmadak | omaralm@kth.se

November 2025

Overview

This report describes our implementation of the Apriori algorithm for mining frequent itemsets and association rules from a transaction dataset. Each transaction is a basket of items represented as a whitespace separated line in a text file. The code first discovers all itemsets whose support is at least a user defined threshold, then generates association rules whose confidence is at least a second threshold.

1 Algorithm and Implementation

Frequent itemsets with Apriori

We treat the input file as a sequence of transactions, where each transaction is a set of item identifiers appearing on one line. The algorithm starts with a first pass over the data in order to count the support of single items and to determine the total number of transactions. From these counts we construct the set of frequent single items, which we call L_1 , by keeping only items whose support is at least the minimum support threshold.

For each level $k \geq 2$ we generate candidate k itemsets C_k from the frequent $(k - 1)$ itemsets L_{k-1} using the join and prune strategy. We sort each $(k - 1)$ itemset into a tuple and join two such tuples if their first $k - 2$ positions are identical. This gives a candidate of size k . We then prune candidates by checking that every $(k - 1)$ subset of the candidate is present in L_{k-1} . Candidates that fail this property are discarded. Once C_k is constructed we scan the dataset once and count how many transactions contain each candidate. From these counts we construct L_k by keeping only candidates whose support meets the threshold. The process stops when L_k is empty. We store all frequent itemsets in a dictionary indexed by k , and for output we represent each itemset as a sorted tuple together with its support count.

Internally we represent itemsets as `frozenset` objects so that equality and membership do not depend on the order of items and to allow sets of itemsets and dictionary keys. For output and sorting we convert them to tuples in a consistent order. This matches the mathematical view of an itemset as an unordered set, while still allowing us to perform prefix checks and join operations easily.

Association rule generation

After we obtain all frequent itemsets we generate association rules. For each frequent itemset I with size at least two we list all non empty proper subsets $X \subset I$. For each such subset we

form a candidate rule

$$X \rightarrow I \setminus X.$$

We compute the support of I and the support of X from the frequent itemsets dictionary and define the confidence of the rule as

$$\text{conf}(X \rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}.$$

We keep rules whose confidence is at least the user supplied threshold. Each rule is stored together with its support and confidence, and the final list of rules is sorted in decreasing confidence and then decreasing support.

2 Program Execution

Datasets

We used the market basket dataset `T10I4D100K.dat`. It contains exactly 100000 transactions. Each line corresponds to one transaction and consists of items separated by whitespace. Our implementation assumes this format. Any other transaction dataset from the web can also be used as long as it has one transaction per line and items separated by whitespace.

How to build and run

The implementation is written in Python and uses only the standard library. We require Python version 3.10 or newer. Optionally we can create and activate a virtual environment using:

```
python -m venv venv
```

The main entry point is the script `apriori.py`. There are two convenient ways to run the program.

Using the Makefile. We set the `DATA` variable in `Makefile` to the path of the dataset file. For example, on Windows we can write

```
DATA="C:\KTH\Data Mining\Data_mining_HWs\Homework2\Dataset\T10I4D100K.dat"
```

We can also adjust `MIN_SUPPORT_PERCENT` and `MIN_CONF` in the `Makefile` if we want to change the support or confidence thresholds. From the `Homework2` folder we then run

```
make run
```

which invokes `python apriori.py` with the chosen parameters.

Direct CLI. We can also run the script directly. For a percentage based support threshold we can use

```
python apriori.py --path "C:\...\T10I4D100K.dat" \  
--min_support_percent 1.0 --min_conf 0.5
```

For an absolute support threshold we can use

```
python apriori.py --path "C:\...\T10I4D100K.dat" \  
--min_support 1000 --min_conf 0.5
```

On Windows the dataset path should be in quotes if it contains spaces.

Command line parameters

The program accepts the following command line parameters.

- `-path PATH`. Required. Path to the transaction dataset file. There is no default.
- `-min_support MIN_SUPPORT`. Optional integer. Minimum support expressed as an absolute number of transactions. If provided, this value is used directly.
- `-min_support_percent MIN_SUPPORT_PERCENT`. Optional float. Minimum support expressed as a percentage of the total number of transactions. The default is 1.0, which corresponds to 1 percent. The program first counts the number of transactions and converts this percentage to an absolute support count. If both `-min_support` and `-min_support_percent` are given, the absolute minimum support takes precedence.
- `-min_conf MIN_CONF`. Optional float between 0 and 1. Minimum confidence threshold for association rules. The default is 0.5.

When the program starts it prints the dataset path and the number of transactions. It then prints the chosen minimum support in both percentage and absolute form. After running Apriori it prints all frequent itemsets grouped by size and their support counts, followed by a timing summary and detailed per level timings. Finally it prints all generated association rules, sorted by confidence.

Results

We evaluated our implementation on the `T10I4D100K.dat` dataset, which contains 100000 transactions. We fixed the confidence threshold at 0.5 and varied the minimum support threshold over three values, 5 percent, 2 percent and 1 percent. For each run we recorded how many frequent itemsets of each size were found and how long the Apriori phase took. The summary is shown in Table 1.

Support	Min. supp.	$ L_1 $	$ L_2 $	$ L_3 $	Total L	Apriori time [s]	# Rules
5%	5000	10	0	0	10	0.78	0
2%	2000	155	0	0	155	83.13	0
1%	1000	375	9	1	385	774.16	7

Table 1: Summary of frequent itemsets and Apriori runtime on `T10I4D100K.dat` for different minimum support thresholds.

We see that at 5 percent support the algorithm finds only 10 frequent single items and finishes very quickly. At 2 percent support the number of frequent single items increases to 155 and the runtime rises to about 83 seconds, but there are still no frequent itemsets of size 2 or larger. At 1 percent support the algorithm finds 375 frequent 1-itemsets, 9 frequent 2-itemsets and 1 frequent 3-itemset, and the Apriori time increases to about 774 seconds. In this last case the rule generator produces 7 association rules that satisfy the confidence threshold.

The detailed per level timings that the program prints show that the time to construct the candidate sets C_k is very small compared to the time to compute the frequent sets L_k . Constructing C_k is an in memory join and prune step on L_{k-1} , while computing L_k requires a full pass over all 100000 transactions and, for each transaction, checking which candidates are contained in it. This explains why the reported C_k times are close to zero, while the L_k times dominate the Apriori runtime.

The large increase in runtime between 5%, 2% and 1% support is due to the rapid growth in the number of candidate 2-itemsets. At 5% support there are only 10 frequent single items and very few candidate pairs, so counting them is fast. At 2% support there are 155 frequent single items, which leads to a much larger C_2 and an expensive L_2 counting step, even though no pairs end up being frequent. At 1% support there are 375 frequent single items, so the number of candidate pairs and the cost of counting them increase further, which explains why the Apriori time grows from less than one second at 5% to about 83 seconds at 2% and to about 774 seconds at 1%.